Corrección Parcial Uno

Colin Torres Ana Ivonne

Pregunta 0.1. Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código ¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia

Una clase es la plantilla que define la estructura y el comportamiento de una entidad. Un objeto es una instancia concreta creada a partir de una clase.

Pregunta 0.2. ¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno

La firma de un método es la combinación única que identifica al método dentro de una clase, está formada por el nombre del método y la lista de parámetros. No incluye el tipo de retorno.

Pregunta 0.3. Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuanto viven.

La variable local se declara dentro del cuerpo de un método o bloque de código; el atributo de instancia se declara dentro de la clase (fuera de cualquier método). La variable local no tiene valor inicial por defecto; el atributo de instancia recibe automáticamente un valor por defecto. La variable local vive únicamente durante la ejecución del método o bloque donde fue declarada; el atributo de instancia vive lo mismo que vive el objeto al que pertenece.

Pregunta 0.4. ¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

null significa que la variable de referencia está vacía y, por lo tanto, no está apuntando a ningún objeto en memoria. Si se intenta invocar un método sobre una referencia que vale null, el programa falla en tiempo de ejecución lanzando una excepción de tipo NullPointerException.

Pregunta 0.5. ¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Conviene usar for-each cuando necesitas leer o procesar todos los elementos de un arreglo de forma secuencial sin preocuparte por límites de desbordamiento. Se prefiere el recorrido con índice(for) cuando requieres modificar los elementos, conocer la posición numérica, recorrer el arreglo de forma inversa, saltar elementos o detener/romper el ciclo bajo condiciones específicas.

Escaneado con CamScanner

R1. ¿Qué imprime?

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Línea / Paso	Variable i	Evaluación / Expresión	Objeto StringBuilder	Salida Impresa
Inicio	—	new StringBuilder("S")	"S"	Ninguna
Iteración 1	1	1/2 = 0	"S0"	S0
Iteración 2	2	2/2 = 1	"S01"	S01
Iteración 3	3	3/2 = 1	"S011"	S011
texto = "S"+2+1	—	"S"+2 → "S2", luego "S2"+1 → "S21"	"S011"	S21
mitad = (double) (7/2)	—	7/2 = 3 (int), luego (double)3 → 3.0	"S011"	3.0

Escaneado con CamScanner

R2.a: ¿Compila, truena o funciona?

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

No compila. La línea que causa el error es: int unidades = medida; Java no permite conversiones implícitas de un tipo de mayor precisión (double) a una de menor precisión (int) debido al riesgo de pérdida de datos.

R2 b: ¿Compila, truena o funciona?

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Compila pero truena. Lo que causa el error es la línea System.out.println(canales[indice]); El arreglo canales tiene tamaño 3, por lo que sus índices válidos son 0, 1 y 2. La variable indice se inicializa con canales.length (que vale 3). Al intentar acceder a canales[3], se rompe el límite del arreglo y Java lanza una excepción de ejecución de tipo ArrayIndexOutOfBoundsException.

R2 c: ¿Compila, truena o funciona?

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Compila e imprime false/true.

Línea 1: La palabra reservada new fuerza la creación de un nuevo objeto String en la memoria (Heap), independiente de la cadena en el String Pool.

Línea 2: El operador == compara referencias de memoria. Como codigo apunta al objeto recién creado y "F2" apunta al literal del String Pool, tienen distintas direcciones y el resultado es false.

Línea 3: El método .equals() compara la secuencia de caracteres (el contenido). Ambos contienen exactamente "F2", por lo que el resultado es true.

Escaneado con CamScanner

R3: Modelado: base, reto y justificación

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

```
public class Multimetro {
    //Atributos
    private final String inventario;
    private String responsable;
    Private static int prestados = 0;

    //Constructor
    public Multimetro(String inventario) {
        this.inventario = inventario;
        this.responsable = "";
    }

    //Método de cambio
    public void prestar(String persona) {
        this.responsable = persona;
        prestados++;
    }

    //Método de consulta
    public String consultarResponsable() {
        return this.responsable;
    }

}
```

Atributo inventario: De tipo String, de instancia (cada aparato tiene el suyo) y final porque el inventario identifica al equipo y nunca cambia.

Escaneado con CamScanner

Atributo responsable: De tipo String, de instancia (cada equipo tiene su propio encargado) y mutable porque cambia con cada préstamo.

Atributo prestados: De tipo int y static porque pertenece a la clase (cuenta global compartida entre todos los aparatos, no por cada uno).

Constructor: Recibe el inventario único para inicializar la constante y asigna el responsable como vacío ("") porque nace sin responsable.

Método prestar(String persona): De instancia, recibe un String, asigna el nuevo responsable e incrementa el contador global prestados.

Método consultarResponsable(): De instancia, no recibe parámetros y devuelve el atributo responsable.

R4. Código con repetición

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número en texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):
40.72 → 4 : 0 → 1 ; 8 → 1
Escribe el método completo. No imprimas ni leas del teclado.

```
static int contarDigitos(int folio) {
    if (folio == 0) {
        return 1;
    }
    int contador = 0;
    while (folio > 0) {
        folio = folio / 10;
        contador++;
    }
    return contador;
}
```

4072 → 4

El argumento folio es 4072 (distinto de 0). El contador comienza en 0.

- Primera iteración: 4072 > 0, 4072 / 10 es 407, contador es 1.
- Segunda iteración: 407 > 0, 407 / 10 es 40, contador es 2.
- Tercera iteración: 40 > 0, 40/10 es 4, contador es 3.

Escaneado con CamScanner

- Cuarta iteración: 4 > 0, 4 / 10 es 0, contador es 4.
- Fin de ciclo: 0 > 0 es falso. El método devuelve el valor final de contador, que es 4.

0 → 1

El argumento folio es 0. Se cumple la condición if (folio == 0). Por lo tanto, el método ejecuta el bloque if y retorna directamente el valor 1 sin entrar en el ciclo while.

- Devuelve 1

8 → 1

El argumento folio es 8 (distinto a 0). El contador comienza en 0.

- Primera iteración: 8 > 0, 8 / 10 es 0, contador es 1.
- Fin del ciclo: 0 > 0 es falso. El método devuelve el valor final de contador, que es 1.

R5. Codiga con selección

Clasifica la carga de un equipo. Usa switch clásico sobre carga/10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

static String nivelCarga(int carga)

Regla o valor recibido	Texto devuelta
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Escaneado con CamScanner

```
static String nivelCarga(int carga) {
    if (carga < 0 || carga > 50) {
        return "INVALIDA";
    }

    switch (carga / 10) {
        case 0:
        case 1:
            return "BAJA";
        case 2:
        case 3:
            return "MEDIA";
        case 4:
        case 5:
            return "ALTA";
        default:
            return "INVALIDA";
    }
}
```

El if (carga < 0 || carga > 50) descarta los límites inválidos primero.

División entera (carga / 10): Al dividir un número entero entre 10 en Java, se truncan los decimales.

Escaneado con CamScanner